

Progressive Retrieval Attention for llama.cpp: Portable Sequence-Attached Semantic K/V in the GGUF Ecosystem

Jorge Simão

September 2026

Abstract

llama.cpp makes quantized local inference portable across CPUs and accelerator backends. Progressive Retrieval Attention (PRA) asks for a different form of portability: independently addressable semantic records whose selected native key/value (K/V) states can be reused without becoming visible prompt tokens. We pin llama.cpp commit `458681e1d5d4a29a1463c4732e03226cf384b997`, implement a PRA SDK adapter, audit public slot and state APIs, and qualify selected-text execution on Apple M4 Pro and M5 CPU/Metal backends. A Qwen3-8B replication on 15 natural-QA examples used 10.0–42.2% of the full-context prompt tokens. Metal median time to first token fell on HotpotQA (696 to 457 ms) and QASPER (2733 to 348 ms), while 2Wiki was confounded by warm prefix reuse. A live slot checkpoint saved 13 positional tokens as 160,572 bytes and restored them in 2.327 ms. The audit shows that this state is sequential prefix state, not detached semantic K/V. However, the public in-process memory API exposes a narrower native seam: encode a resource under one sequence, enable unified K/V storage, and attach its existing cells to a request with `llama_memory_seq_cp`. Across five prompts on CPU and Metal, all 10 schedule-matched E0/E2 runs have zero maximum logit error through four decode steps. We therefore qualify HTTP/slot execution at E0 and this pinned in-process, prefix-positioned mechanism at E2.

1 Introduction

Local inference has two kinds of context pressure. The first is computational: long prompts increase prefill work and K/V residency. The second is semantic: applications need records, documents, tool outputs, and task state that do not naturally form one immutable prefix. llama.cpp handles the first problem with portable GGUF execution, quantization, slots, and cache controls. PRA addresses the second by separating a logical record from the small region selected for a particular query.

The integration question is not whether a cache can be saved. It is whether an engine can attend to selected K/V from an independently named resource while preserving ordinary query positions, masks, and a single normalization over direct and memory keys. Confusing a restored conversation prefix with such memory would produce an attractive but incorrect E2 claim.

This paper contributes: (i) a capability-honest llama.cpp SDK provider and adapter; (ii) a pinned audit of public server and cache-state seams; (iii) live CPU and Metal E0 qualification with frozen evidence selections; and (iv) a minimal native selected-K/V implementation over llama.cpp’s public C API with schedule-matched logit and persistent-decode parity.

2 PRA Contract

For query state q and selected native memory (K_M, V_M) , native PRA requires one attention normalization,

$$\text{Attn}(q) = \text{softmax}\left(\frac{q[K_D; K_M]^\top}{\sqrt{d}} + M\right) [V_D; V_M], \quad (1)$$

where (K_D, V_D) is direct sequential context and M encodes the correct causal and resource mask. Concatenating decoded memory text is E0. Restoring a saved sequential cache is prefix reuse. Neither is native E2.

We use four evidence levels:

E0 selected source appears as ordinary visible text;

Typed PRA records → authorization and frozen selection → **E0**: selected text in llama-server prompt.
 The same selection → resource sequence in unified K/V → metadata-only sequence attachment → **E2**: query-only decode over selected native K/V.
 llama.cpp independently owns GGUF loading, CPU/Metal/CUDA/Vulkan placement, slots, scheduling, graph execution, and kernels. Slot snapshots remain on the ordinary sequential-state path.

Figure 1: Ownership and integration boundary. E0 is measured through the HTTP server. The narrow E2 path is measured through the pinned in-process C API.

E1 logical resource identity survives transport, but attention remains text/prefix based;

E2 the model consumes detached selected native K/V;

E3 the engine scheduler owns PRA placement, prefetch, sharing, and eviction.

3 Pinned llama.cpp Audit

The pinned source was built with CMake 4.4.3 and Ninja 1.13.2. The resulting `llama-server` exposes OpenAI-compatible chat, explicit slots, unified KV configuration, cache quantization, CPU execution, and Metal offload. GGUF owns model topology and quantized weights; llama.cpp owns tensor placement, graph execution, attention kernels, and sequential cache representation.

The public slot save/restore endpoint is operational only when the server is launched with `--slot-save-path`. Its object is an engine slot: token positions, recurrent conversational state, and ordinary cached K/V. It does not accept an immutable resource identifier, per-layer selected intervals, or separate direct/memory geometry. Consequently, slot state cannot be used to claim E2.

The in-process C API has a distinct seam. `llama_memory_seq_cp` adds one sequence’s cache cells to another sequence. With `kv_unified=true`, the pinned implementation updates cell metadata and does not copy K/V buffers. A selected resource can therefore remain under a stable resource sequence, be attached to a request sequence, and be consumed without resubmitting its text. This is native transport with ordinary prefix-equivalent positions; it is not yet an arbitrary-position memory branch.

4 Implementation

`LlamaCppEngineAdapter` exposes the OpenAI-compatible path and reports E0. An optional `LlamaCppSlotClient` derives checkpoint filenames from the model fingerprint, tenant, session, resource URI/version, and source hash. This prevents an application from reusing a slot solely because two visible prompts look alike. The filename discipline improves prefix-state isolation but does not upgrade its integration level.

Only an explicit `LlamaCppNativeExecutor` can advertise E2. It receives the versioned `PRAWireRequest` and a `LogicalPRABlockStore`. The reference probe implements its engine-side core in C++: encode the selected resource under a dedicated sequence; attach those cache cells to the request; then decode query tokens at the same logical positions used by split E0. Unit tests enforce that HTTP and slot paths remain E0, that resource changes alter slot identity, and that only a native executor changes the advertised capability. Production wiring must additionally bind sequence IDs to tenant, model, resource version, and request lifetime.

5 Experimental Method

We used an Apple M4 Pro with 14 CPU cores, 20 GPU cores, and 48 GB unified memory. The model was Qwen2.5-0.5B-Instruct in Q4_K_M GGUF. Metal used `-ngl 99`; CPU used `-ngl 0`. Both used a 4096-token context and four slots. The natural benchmark freezes the selected evidence before engine execution and compares no context, E0 selected text (up to 384 source tokens), and full context (up to 1024 source tokens). Metal covers five examples from each of HotpotQA, QASPER, and 2WikiMultiHopQA with two repeats. CPU is a three-example-per-dataset backend smoke test.

Quality is token F1 on short generated answers. System metrics are measured from streaming HTTP responses. The small model, short generation cap, and small cohort make quality directional rather than conclusive. CPU and Metal cohorts also differ in repeat count; their absolute quality is not paired.

Table 1: Metal E0 qualification. Prompt ratio is selected/full. TTFT is median milliseconds; F1 is selected/full.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.410	37.0 / 60.1	0.314 / 0.314
QASPER	0.375	31.0 / 58.9	0.243 / 0.077
2Wiki	0.517	35.6 / 27.3	0.029 / 0.129

Table 2: CPU backend smoke test. The three-case cells are calibration evidence, not workload-scale estimates.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.411	305.5 / 530.5	0.103 / 0.190
QASPER	0.356	225.7 / 534.6	0.070 / 0.106
2Wiki	0.525	256.8 / 275.5	0.000 / 0.000

We then ran a larger-model replication on an Apple M5 with 10 CPU cores, 10 GPU cores, and 16 GB unified memory. It used the official Qwen3-8B Q4_K_M GGUF with the official llama.app binary (build 10679, commit 50f068fff). Selection was again frozen before execution. The Metal cohort used five unique examples per dataset, two repeats, a 384-token selected cap, and a 4096-token full-context cap. A one-example-per-dataset CPU run with four threads is reported only as a backend diagnostic; it is not a matched throughput comparison.

The native mechanism cohort uses pinned llama.cpp commit 458681e1d5d, Qwen2.5-0.5B-Instruct Q4_K_M, and five factual prompts. Each prompt runs on CPU and Metal. The E0 control encodes resource and query in two calls under one request sequence. E2 encodes the same resource tokens under a resource sequence, attaches them to the request in unified-cache mode, and decodes only the same query tokens. We compare every vocabulary logit at the first prediction and through four greedy decode steps. A one-call FULL control is retained separately because changing prefill batch shape can introduce ordinary floating-point differences.

6 Results

The robust result is input reduction, not universal quality or latency improvement. Selected prompts occupy 35.6–52.5% of the full prompt across the two backend runs. Metal selected-text TTFT improves for HotpotQA and QASPER, but 2Wiki reverses direction because prefix-cache state and a short full prompt dominate this small run. Selected evidence preserves HotpotQA F1 on Metal and raises QASPER F1, while 2Wiki loses quality. This is consistent with an upstream selection limitation, not evidence for or against native representation.

The 8B replication strengthens the systems observation without turning the smoke cohort into a quality claim. Selected text reduced the visible prompt by 57.8–90.0%. It reduced median TTFT by 34.3% on HotpotQA and 87.3% on QASPER. The reversed 2Wiki median coincides with 682 mean cached tokens for the full condition versus 207 for selected text; its full-context p95 was still 1931 ms versus 860 ms selected. Thus warm prefix reuse can dominate a tiny median even while selected text improves the tail. Quality was approximately preserved on 2Wiki, improved on these five Hotpot examples, and changed only slightly on QASPER. These signs are descriptive because examples, not repeated requests, are the independent units.

The four-thread CPU diagnostic was orders of magnitude slower: selected TTFT was roughly 37–39 s and full-context TTFT 45–68 s across the three cases, whereas Metal produced about 24 output tokens/s. This establishes accelerator placement as operationally necessary for this 8B local workload; the CPU sample is too small for a backend-quality comparison.

6.1 Slot-state qualification

A live slot operation saved 13 cached tokens, wrote 160,572 bytes in 0.412 ms, and restored the same 13 tokens from 160,572 bytes in 2.327 ms. This is useful for repeated sequential sessions. It also demonstrates why slots are not a substitute for PRA: the saved object is much larger than a logical reference, is position-shaped, and has no selected-resource attention contract.

Table 3: Qwen3-8B Q4_K_M replication on M5 Metal (five examples per dataset, two repeats). Prompt ratio is selected/full; TTFT is p50 milliseconds.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.327	457 / 696	0.381 / 0.181
QASPER	0.100	348 / 2733	0.185 / 0.150
2Wiki	0.422	452 / 247	0.253 / 0.255

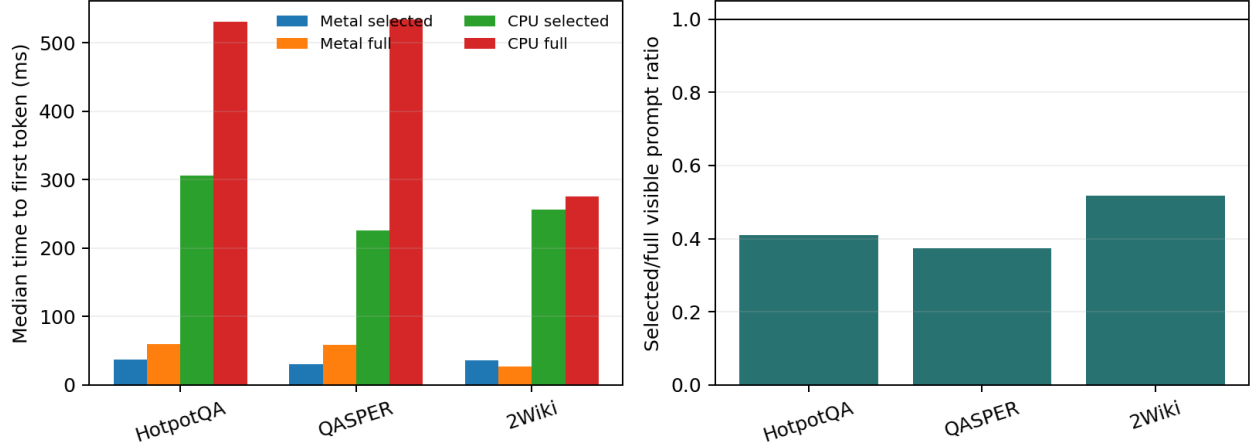


Figure 2: Measured E0 context-pressure qualification. Selected text reliably reduces visible tokens; latency and quality effects remain workload dependent.

6.2 Native sequence-attachment qualification

Schedule-matched E0 and E2 have maximum logit error exactly zero in all 10 runs, including every persistent-decode step. One-shot FULL retains the same top token in 10/10 runs but differs numerically from split E0/E2: maximum FULL-E2 error ranges up to 0.415 on CPU and 0.0201 on Metal. The split control localizes that difference to prefill batch shape rather than native sequence attachment. In unified-cache mode, upstream `seq_cp` adds the request sequence ID to existing cells; it does not copy their physical K/V payload. After first-request cleanup, absent second requests match a fresh query exactly on CPU. Metal preserves all five top tokens but has maximum logit differences of 0.0015–0.0058 because resident masked cells change the unified-cache kernel shape. Explicitly reattaching the still-resident resource reproduces first-request logits exactly in all 10 runs.

7 Capability and Stop Gate

The mechanism gate is closed at narrow scope: E0 works, slot state is precisely classified, and unified-cache sequence attachment has exact CPU/Metal logits and persistent decode. Server request plumbing, multi-resource controls, physical memory accounting under load, CUDA/Vulkan replication, and E3 scheduler ownership remain open.

8 Limitations and Next Work

The cohorts remain small; the 8B replication broadens model scale but not the number of independent questions. Native parity uses five synthetic factual prompts and one model, not natural QA. We do not measure gold-answer log probability, physical sharing under concurrency, server tail latency, or multi-resource budgeting. The sequence-attached mechanism also preserves prefix-equivalent positions; arbitrary non-prefix positional geometry would still require a graph-level attention extension. Next work is HTTP request wiring with absent/wrong/multiple-memory and tenant-isolation controls, followed by matched CPU, Metal, CUDA, and Vulkan runs.

Table 4: Pinned in-process native selected-K/V mechanism cohort. Exact logits refer to schedule-matched split E0 versus E2 through four decode steps.

Backend	Cases	Exact logits	Exact decode	Isolation	Warm reuse
CPU	5	5/5	5/5	5/5 exact	5/5
Metal	5	5/5	5/5	5/5 < .006	5/5

Table 5: Qualification by backend and level at the pinned revision.

Backend	E0	E2	E3
CPU	measured (0.5B; 8B diagnostic)	5-case in-process mechanism	not implemented
Metal	measured (0.5B, 8B)	5-case in-process mechanism	not implemented
CUDA	not run here	not run	not run
Vulkan	not run here	not run	not run

9 Conclusion

llama.cpp supplies a portable selected-text baseline and cheap sequential slot persistence. Slot restore remains E0/E1. The pinned public C API nevertheless supports a narrow E2 path: unified-cache sequence membership can expose an already encoded selected resource to a request with exact schedule-matched logits and persistent decode. This result is an implementation foothold, not a finished server product. Resource identity, authorization, lifecycle, concurrency, multi-resource geometry, and arbitrary non-prefix positions still belong in the adapter and scheduler layers.

Artifact Availability

The adapter, tests, raw JSON, figure generator, commands, and manuscript are available on the Paper 6.7 research branch. The README records the exact branch, pinned model, backend flags, and benchmark invocation.

References

- [1] G. Gerganov et al. *llama.cpp*. 2023–2026. <https://github.com/ggml-org/llama.cpp>
- [2] llama.cpp contributors. *GGUF format documentation*. <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
- [3] Qwen Team. *Qwen2.5 Technical Report*. 2024. <https://arxiv.org/abs/2412.15115>